

Vector Compression & Memory/Storage Economics FP16 Vectors and Matryoshka Embeddings

Data Engineer
SQL Community Volunteer
Lifelong geek
Do tech to hike



Environment



Dataset :
HackerNewsStories – 3.8
million rows



Model : nomic-ai/nomic-
embed-text-v1.5 -
Opensource model



Python based code to
generate embeddings



Storage & Query Engine:
Microsoft SQL Server 2025
(RTM-CU7) (KB5096981) -
17.0.4065.4 (X64)

Key Terms (At a Glance)



Model

A trained system that turns text into vectors.



Embeddings

The vectors produced by a model that capture meaning.



Dimensions

The number of values in a vector.



Vector

A list of numbers that represents meaning.



Cosine distance

A way to measure how similar two vectors are.



Semantic Space

A space where similar meanings are close together.



Prefix

The first few dimensions of a vector.



DiskANN

A fast search algorithm for large vector collections on disk.

GOALS

Reducing vector size without losing semantic fidelity

How?

What drives the cost of vector search?

Do I need the precision?

Do I need all the dimensions?

How can I evaluate quality?

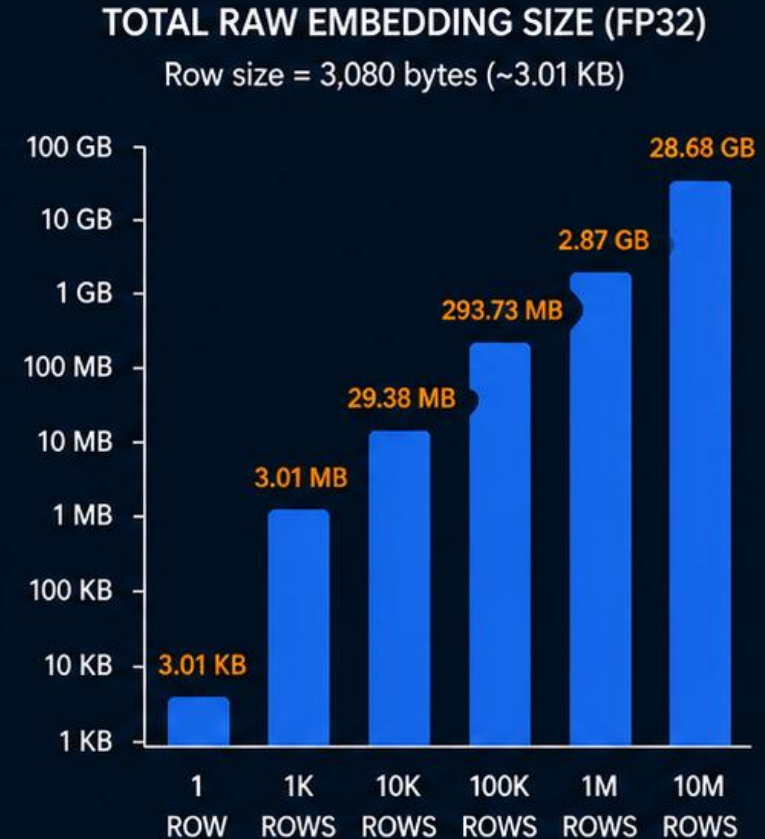
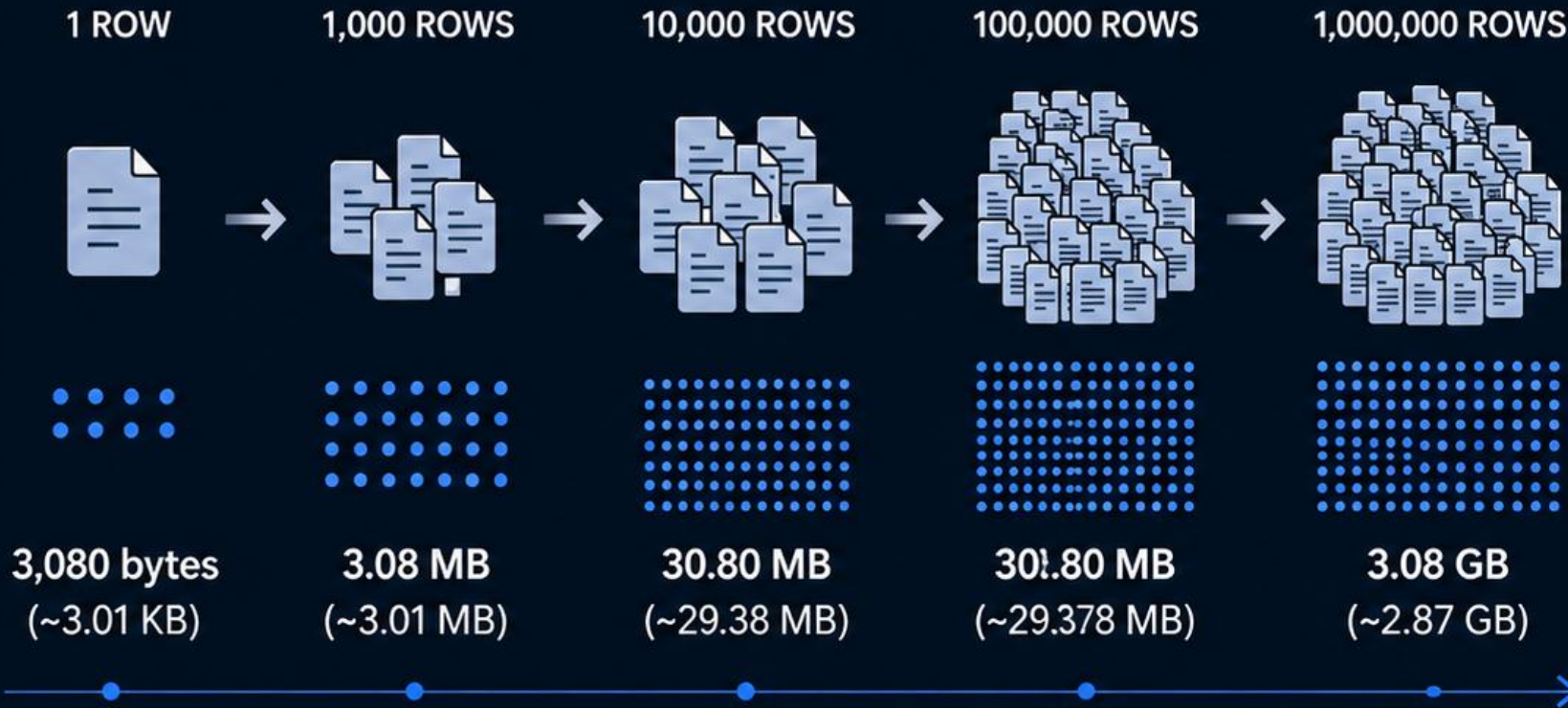
What are the tradeoffs?

Demo : Data used
and what is Vector
search

FP32 STORAGE SCALING: FROM 1 ROW TO MILLIONS

768 dimensions • 4 bytes per value

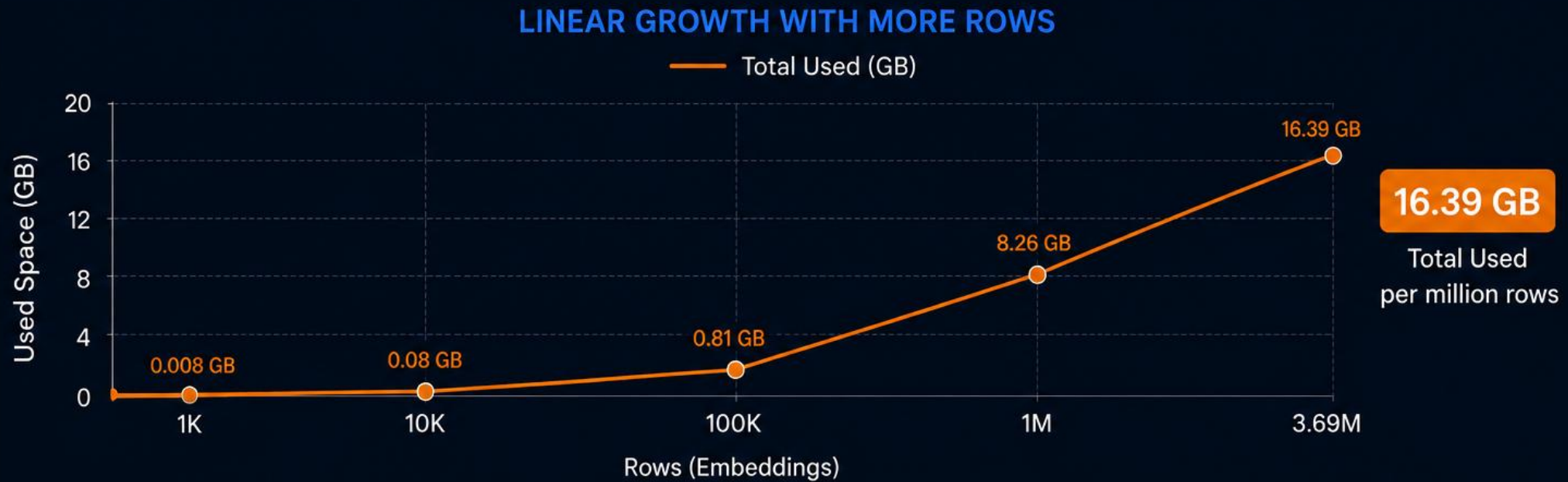
Row size = 3,080 bytes (~3.01 KB)



TAKEAWAY: FP32 → **3.08 GB** per million rows. **Storage grows linearly.**

SCALABILITY IS MORE THAN SPACE

As embeddings grow, the impact touches every part of the system



AREAS IMPACTED



STORAGE

More rows means more disk, memory, and cache consumption.



HIGH AVAILABILITY

More data means more to synchronize between replicas, increasing network and replication lag.



DISASTER RECOVERY

More data to move and more time to restore during failover or recovery.



BACKUPS

Bigger backups, more time to backup, and more storage required.



PERFORMANCE

More data leads to more I/O, higher cache pressure, and slower queries.



The cost of data growth is paid everywhere: storage, availability, recovery, backups, and performance.

Smaller representations reduce the impact across the entire lifecycle.



Can we make
every vector
smaller
without breaking
semantic search?

Vector Representation Choices

1 FP32



Full Precision (Baseline)

 Precision 32-bit float

 Dimensions 768

 Size / Vector ~3.0 KB

 Use case
Highest quality (baseline)

2 FP16



Half Precision

 Precision 16-bit float

 Dimensions 768

 Size / Vector ~1.5 KB (50%)

 Use case
Lower storage, similar quality

3 Matryoshka 512



First 512 Dimensions

 Precision 16-bit float

 Dimensions 512

 Size / Vector ~1.0 KB (67%)

 Use case
Balanced size and quality

4 Matryoshka 256



First 256 Dimensions

 Precision 16-bit float

 Dimensions 256

 Size / Vector ~0.5 KB (83%)

 Use case
Maximum compression

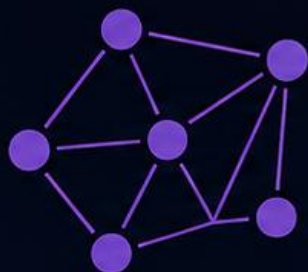
How Do We Evaluate Our Choices?

*FP32 (768 dims) is our **baseline***

1 RETRIEVAL FIDELITY



2 GEOMETRY



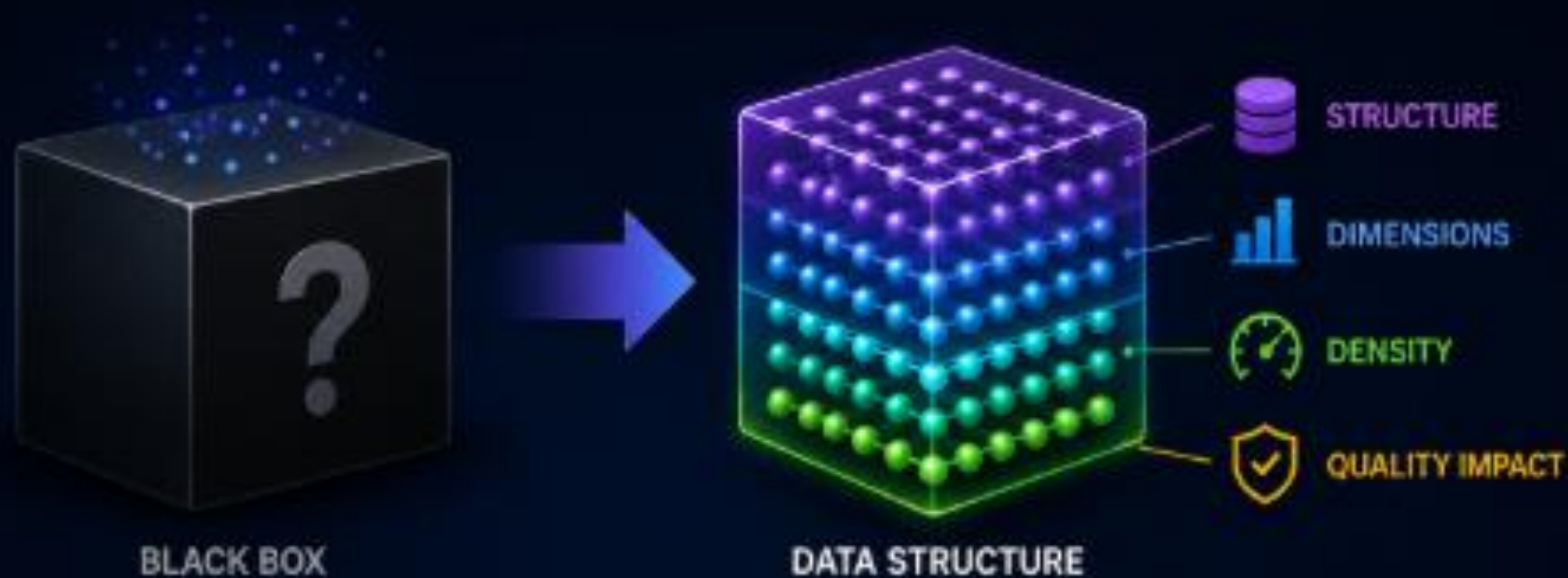
3 LATENCY



4 STORAGE



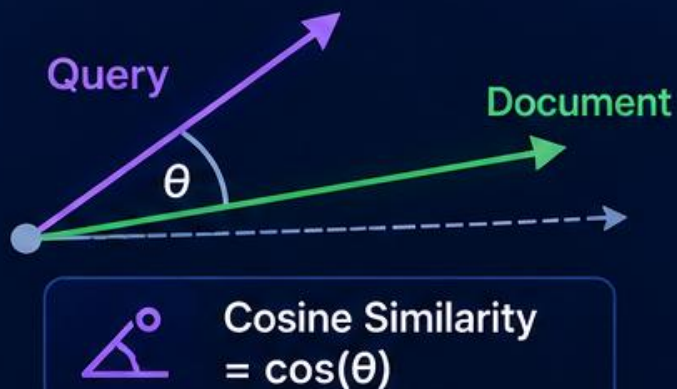
Treat embeddings as **data structures**, not **black boxes**.



From Similarity to Geometry

Similarity in vector search is the projection of geometry.

Similarity



Geometry

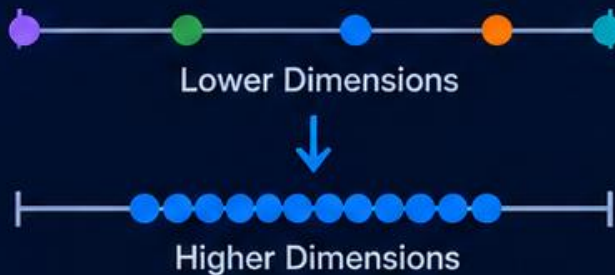


3 Key Properties of Embedding Space

1 Semantic Neighbourhoods



2 Distance Concentration



3 Hubness



Semantic Neighborhood

Good neighbors should be close together.



We take a query item and look at its nearest neighbors.

The results are **semantically related** to the query, even if the wording is different.



LIBRARY EXAMPLE

Query book: "The Hobbit"



HOW TO READ

- Very close
(very similar)
- Close
(similar)
- Moderately close
(some relation)
- Far
(not related)

DISTANCE CONCENTRATION

More noodle categories = items look more similar,
harder to find the true nearest item.

HOW TO READ

- Very close
- Close
- Moderately close
- Far

1 FEWER CATEGORIES

(Broad categories)



Distances are very different.
Closest item is easy to identify.

2 MORE CATEGORIES

(More specific noodle types)



Distances are closer together.
Harder to pick the closest item.

3 MANY CATEGORIES

(Very specific noodle variations)



Distances are very similar.
Harder to find the true nearest item.



KEY TAKEAWAY:

More specific = more similar.
Harder to find the one.

LIBRARY EXAMPLE

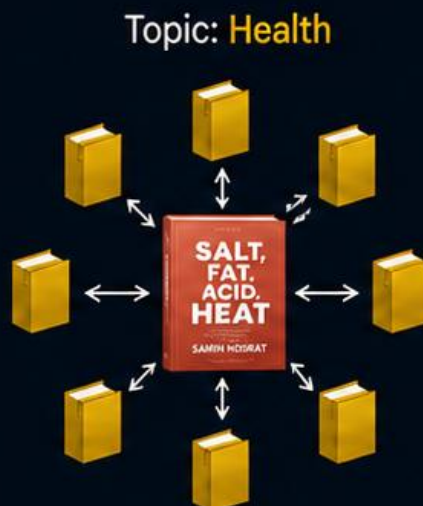
HUBNESS



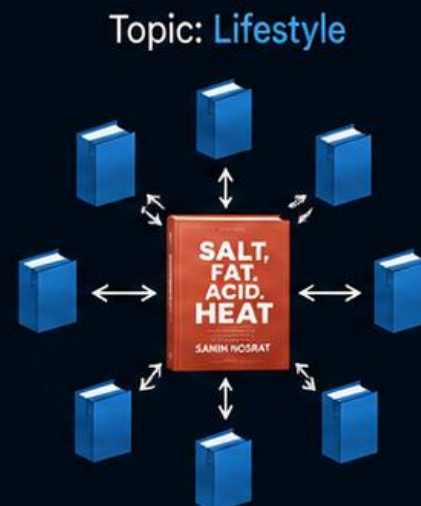
Same few items appear in many different neighborhoods.



Appears in many **cooking** recommendation lists.



Appears in many **health** recommendation lists.



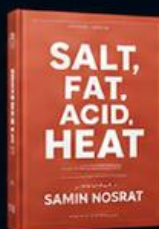
Appears in many **lifestyle** recommendation lists.

WHY DOES THIS HAPPEN?

- ✓ Book shares similarity with many topics.
- ✓ Not the best match for most.
- ✓ But often “good enough” to appear.

➔ Result: It becomes a **hub** — appears in many neighborhoods.

EXAMPLE BOOK



Fits in many topics



Cooking



Health



Lifestyle



Personal Growth



Finance



Travel

...



KEY TAKEAWAY



Few “hub” books appear in many neighborhoods.



This reduces diversity — same popular books repeat.



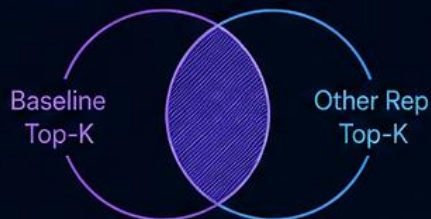
This hurts recommendation variety and discovery.

Key Metrics for Evaluating Embedding Geometry

Ensure compressed embeddings preserve the same search behavior as the baseline.

4 Core Metrics

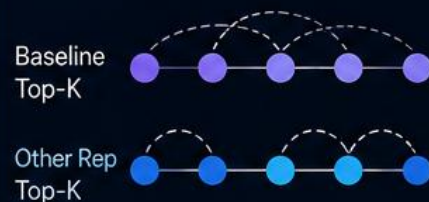
1 Recall / Total Overlap



Average Top-K
Overlap %



2 Neighborhood Preservation



Average Overlap
Count



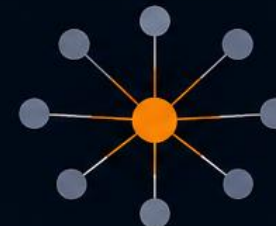
3 Distance Concentration



Min / Max Overlap
Count



4 Hubness (k-Occurrence)



Exact Match
Query %



3 Nice-to-Have Metrics

5 Mean Distance Ratio (Rank-1 / Rank-K)



Average Rank-1 / Rank-K
Distance Ratio



6 Mean Absolute Rank Shift



Average Absolute
Rank Shift



7 Coverage (Unique Neighbor Ratio)



Unique Neighbor
Ratio



Higher is better



Lower is better

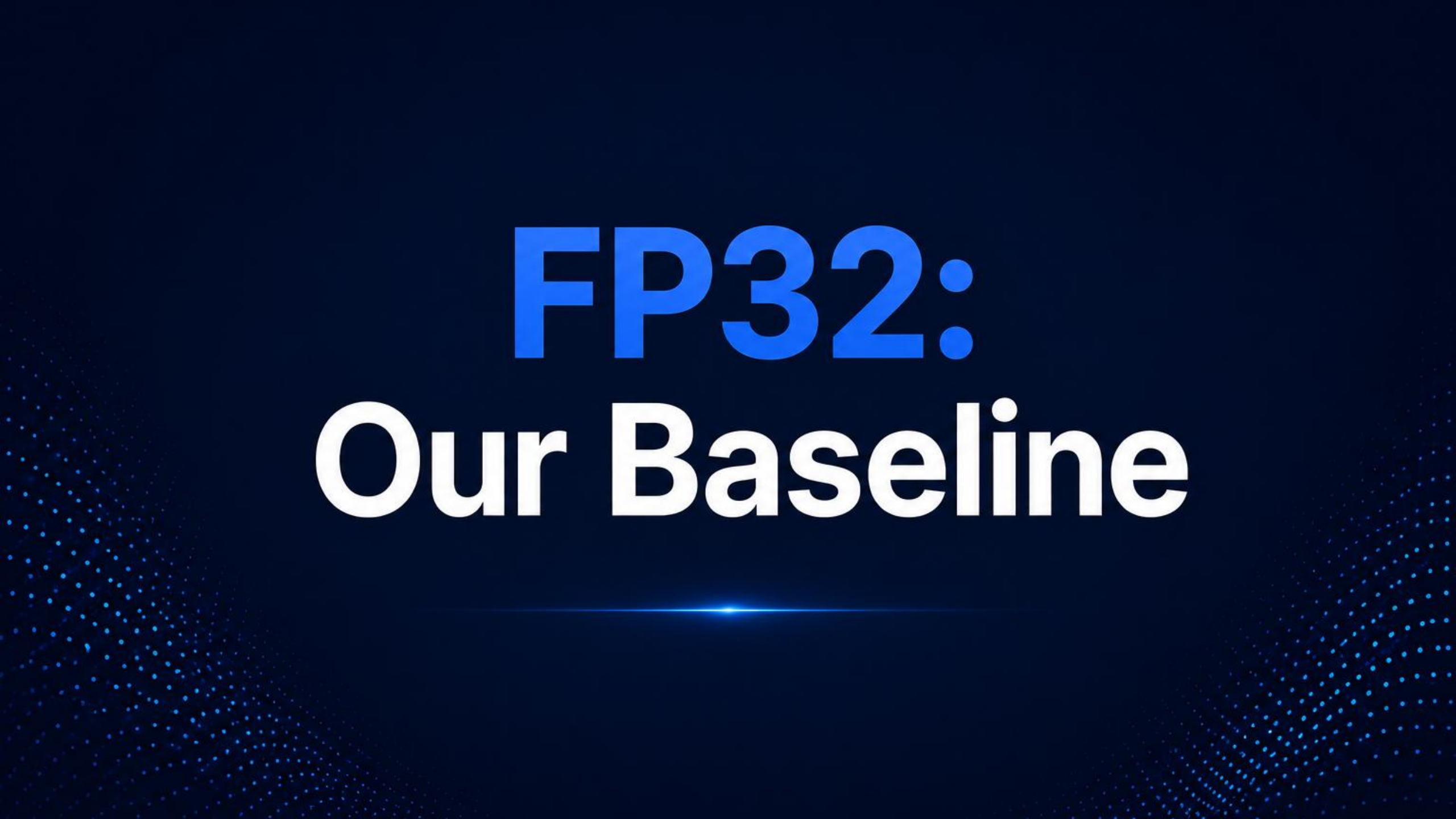


Compares Top-K results



Aggregated across queries

FP32: Our Baseline



SEMANTIC NEIGHBORHOOD SUMMARY (FP32)

Demonstrating Vector Search Results Across 10,000 Queries



Each query returns its top 10 nearest neighbors.

The neighbors are consistently close in vector space (lower cosine distance is better).



QUERIES EXECUTED

10,000

Total Queries



NEIGHBORS PER QUERY

10

Average Neighbors
per Query



NEIGHBOR SELECTIONS

100,000

Total Neighbor
Selections



AVERAGE COSINE DISTANCE

0.1862

Across All Neighbor
Selections

DISTANCE OVERVIEW (FP32)



Average Closest (Rank 1) Distance

0.1378



Average Farthest (Rank 10) Distance

0.2073



Spread (Rank 10 – Rank 1)

0.0695

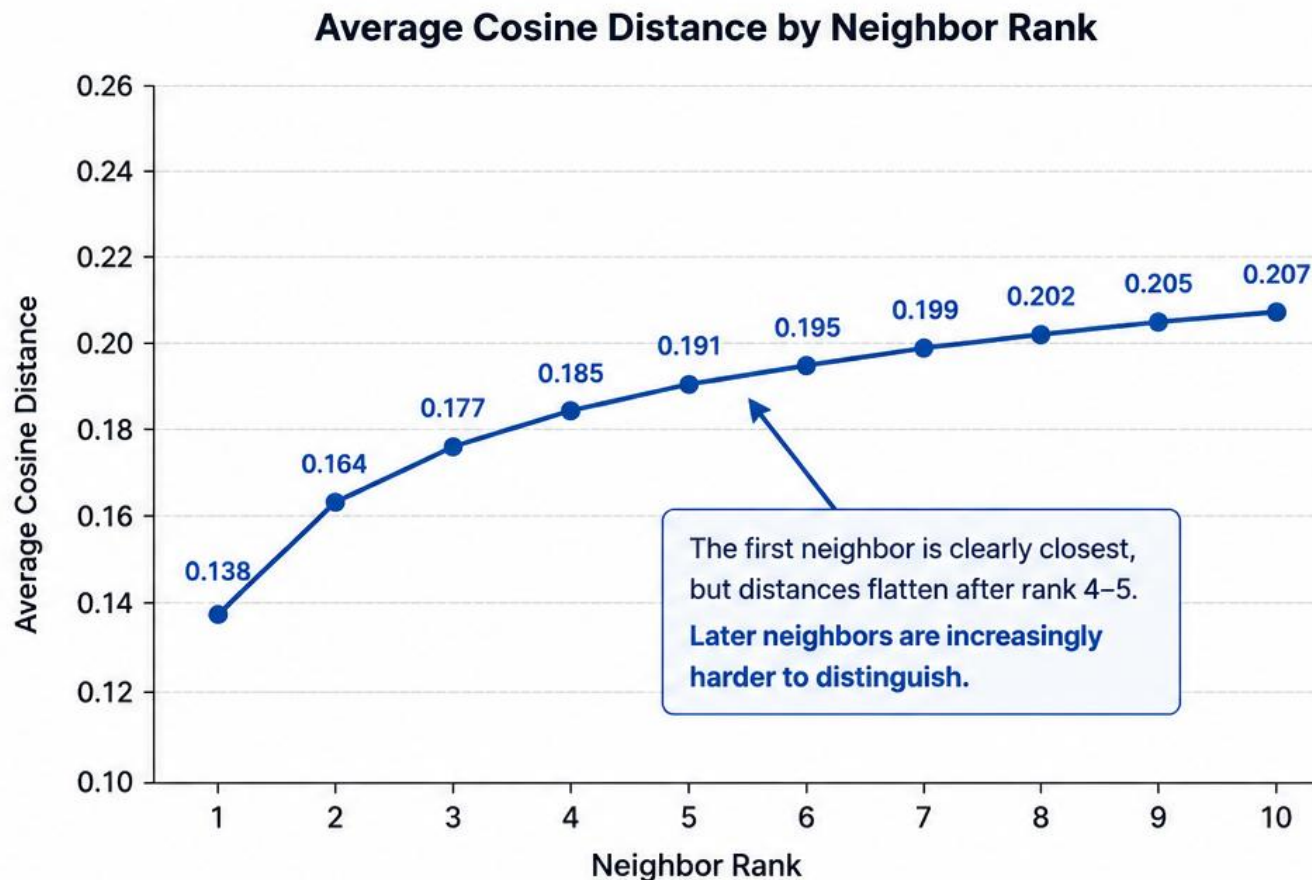


Neighbors are consistently
close to the query,
indicating a strong
semantic neighborhood.

Distance Concentration Across 10,000 Searches

Average Cosine Distance by Neighbor Rank

Neighbor Rank	Average Cosine Distance
1	0.138
2	0.164
3	0.177
4	0.185
5	0.191
6	0.195
7	0.199
8	0.202
9	0.205
10	0.207



Key Takeaway: Across 10,000 searches, the average distance from Rank 1 (0.138) to Rank 10 (0.207) increases only slightly. This gradual flattening is the beginning of distance concentration, making it harder to separate good neighbors from the rest.

Rank 1 Avg. Distance = 0.138
Rank 10 Avg. Distance = 0.207
Rank 1 is 66.5% of Rank 10
(0.138 / 0.207)



HUBNESS

MAX HUB APPEARANCES

4



Most “popular” vector appeared in **4** different neighborhoods

SELECTION FREQUENCY DISTRIBUTION

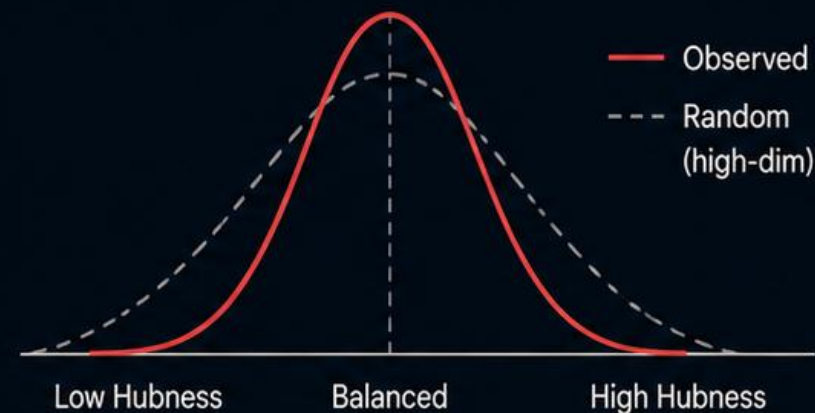


- High (≥ 3)
2.3%
- Medium (2)
15.4%
- Low (1)
32.6%
- None (0)
49.7%



High hubness is low —
no single vector dominates.

HUBNESS (SKEW OF k-OCCURRENCES)



Hubness is low and well-controlled.
Results are not dominated by hubs.



0.3%

Of vectors are
hubs (≥ 3)



4

Max hub
appearances



Low

Overall
hubness



Balanced

Distribution of
k-occurrences



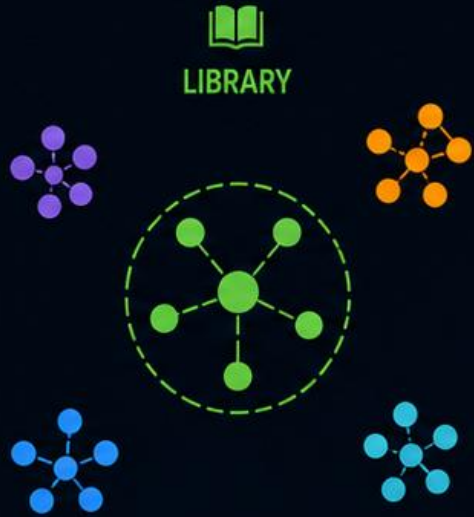
LOW HUBNESS

No single vector dominates the search results.

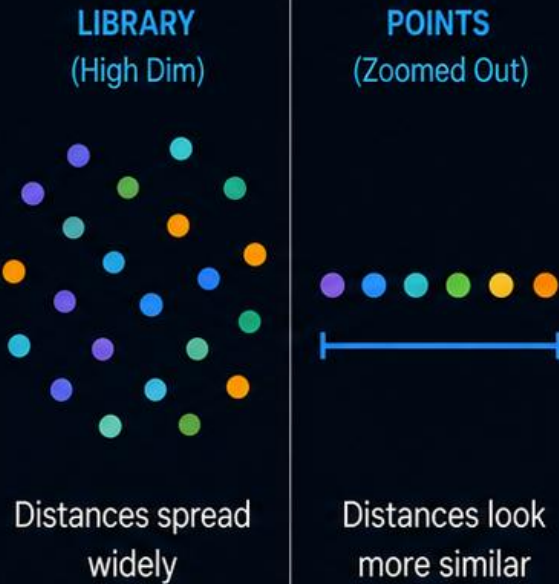


From These Concepts to the Curse of Dimensionality

1 SEMANTIC NEIGHBORHOOD



2 DISTANCE CONCENTRATION



3 HUBNESS



4 CURSE OF DIMENSIONALITY



! THE CURSE OF DIMENSIONALITY



Harder to define neighborhoods

Distances become indistinguishable

Hubs dominate results

Overall search quality drops

IMPACT AS DIMENSIONS GROW



Less accurate retrieval



Less diverse neighbors



More compute for little gain



Higher storage and cost

Why this matters?

Two key reasons drive embedding optimization.

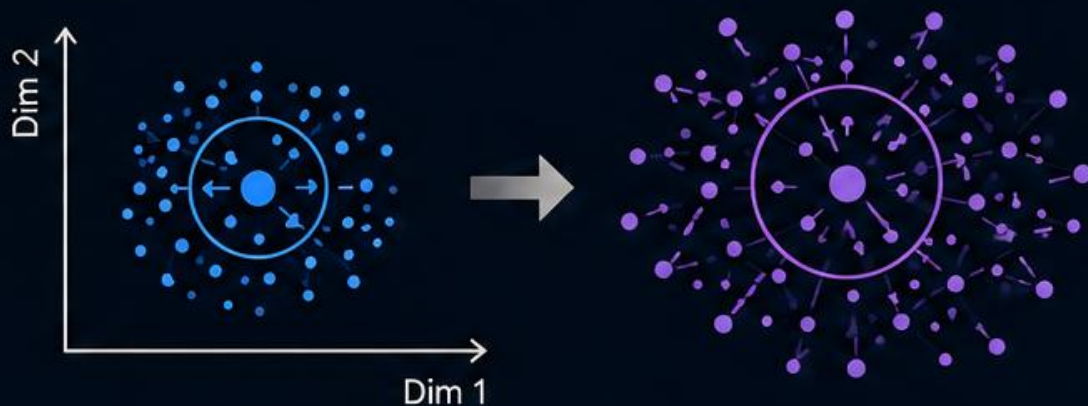
1 Resource Consumption



-  Storage
-  Memory
-  I/O
-  Backup
-  Cost

→ Embeddings are large.
Every **byte** matters.

2 Curse of Dimensionality



Lower dimensions
Distances vary

Higher dimensions
Distances look alike

→ Harder to find the best match.
Search quality can drop.



Our Goal



Reduce
resource use



Preserve semantic quality

FP16:

Reduced precision

FP16 vs FP32 – Fidelity Dashboard

Top-10 Neighbor Overlap Analysis • 10,000 Queries



FP16-768

VS

FP32-768



COMPARISON

QUERY COUNT



10,000

queries



OVERALL FIDELITY

99.98%

Average Top-10 Overlap

AVERAGE OVERLAP COUNT (OUT OF 10)



9.9982

neighbors

AVERAGE TOP-K OVERLAP PERCENT



MINIMUM OVERLAP COUNT



9

neighbors

MAXIMUM OVERLAP COUNT



10

neighbors

EXACT MATCH QUERY COUNT



9,982

queries

EXACT MATCH QUERY PERCENT



DISTANCE CONCENTRATION SUMMARY (TOP-10 NEIGHBORS)

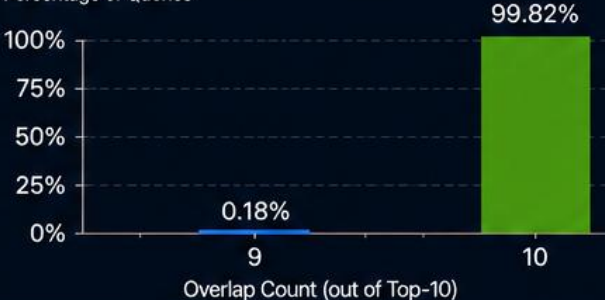


SIMILAR

FP16-768 and FP32-768 show identical distance concentration metrics.

OVERLAP COUNT DISTRIBUTION

Percentage of Queries



HUBNESS SUMMARY (TOP-10 NEIGHBORS)

Maximum Occurrences (Higher = More Hubness)

FP16-768

4

FP32-768

4



FP16 and FP32 show identical maximum occurrences indicating similar hubness characteristics.



KEY TAKEAWAY

FP16-768 preserves the embedding space almost identically to FP32-768, with equivalent overlap and distance concentration metrics, and similar hubness characteristics.



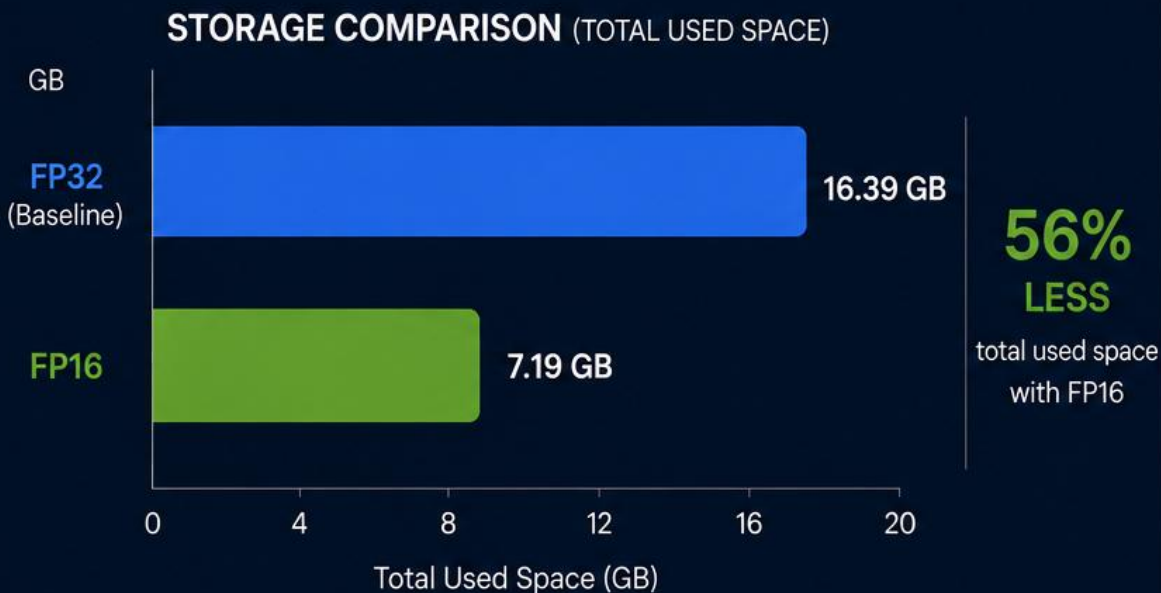
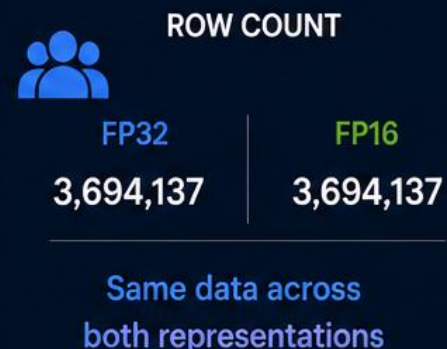
FIDELITY STATUS



EXCELLENT – NEAR IDENTICAL

FP32 vs FP16 – Vector Storage & Index Utilization

Same Data • Same Vector Index • Higher Compression with FP16



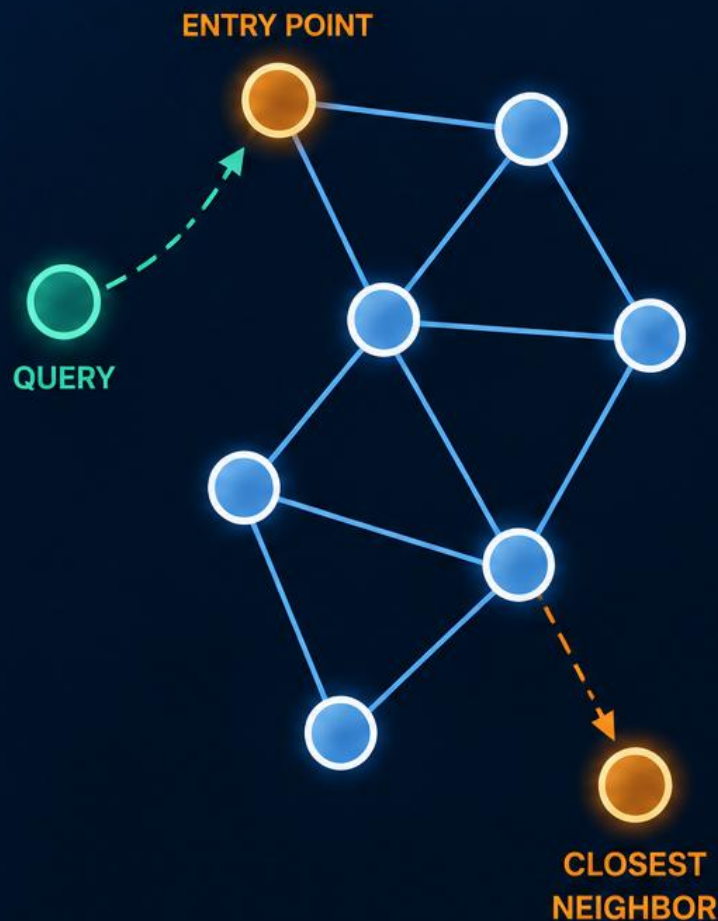
KEY TAKEAWAYS

-  FP16 reduces total database size by **56%** with the same vector index.
-  Embedding storage (table) is **60% smaller** with FP16.
-  Vector index size remains **identical** (0.94 GB) across both representations.

VECTOR INDEX

WHY DOESN'T THE VECTOR INDEX GET SMALLER

HOW IT WORKS: DISK-ANN ALGORITHM



1

Start at entry point

Begin the search from the designated entry point.

2

Check neighbors

Examine neighbors:
Which is closest to the query?

3

Move to closest neighbor

Move to the closest neighbor found.

4

Repeat

Repeat steps 2–3 until local minimum is reached.

5

Closest neighbor

Return the closest neighbor as the search result.

Why FP16 may cause more reads

1 Precision Differences

FP16 quantization slightly changes vector values.

FP32

0.123456	0.987654	-0.543210	...
----------	----------	-----------	-----

FP16

0.123413	0.987549	-0.543457	...
----------	----------	-----------	-----



Lower precision causes tiny numeric differences.

2 Distance Ordering Changes

Distances from the same node get reordered.

Neighbor	FP32 Distance	FP16 Distance
A	0.4078752	0.4078856
B	0.4078767	0.4078752
C	0.4078856	0.4078921
...	...	...



Neighbor order changes (B closer than A in **FP16**, but not in **FP32**).

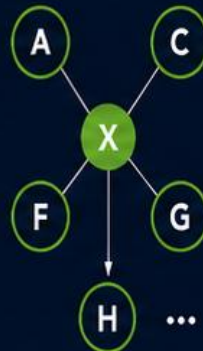
3 Different Graph Topology

Each precision builds a different adjacency (neighbor) list.

FP32 Graph



FP16 Graph



Different adjacency (neighbor) lists create a different graph.

4 Traversal Path Diverges

Searching from the same entry point reaches different nodes.

FP32 Traversal Path



FP16 Traversal Path



Paths diverge, so FP16 explores different parts of the graph.

5 More Work, More Reads

More nodes expanded means more adjacency lists read.

FP32

Nodes Expanded	11
Neighbors Examined	66
New Candidates	29

FP16

Nodes Expanded	15
Neighbors Examined	90
New Candidates	41



More exploration work leads to more adjacency lists read (more **logical reads**).

Why can't we stop here?

MATRYOSHKA 512

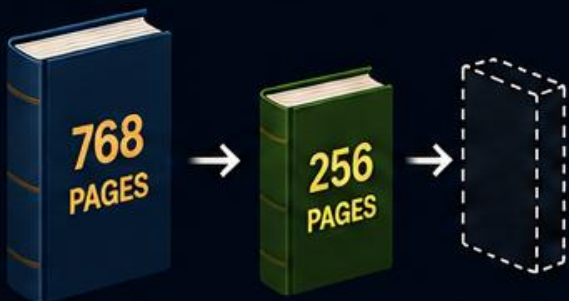
- Compression attempt 1

Compression Strategies



1

NAÏVE TRUNCATION



2

PCA



3

AUTOENCODER



4

MATRYOSHKA



Strategy	Pros	Cons
1 Naïve Truncation	✓ Simple	✗ Unpredictable quality
2 PCA	✓ Captures variance	✗ Not trained for search
3 Autoencoder	✓ Reconstructive	✗ Requires training
4 Matryoshka	✓ Multi-length fidelity	✗ Needs specific model

Matryoshka Aware vs Unaware Models

Same objective: smaller vectors. Very different training.

✓ MATRYOSHKA AWARE

Trained so **every prefix is meaningful**.

Full Vector (768 dims)



Prefix (512 dims)



Prefix (256 dims)



Prefix (128 dims)



**ALL PREFIXES
WORK WELL**



Trained with
multi-length
objective



Consistent
quality at
all sizes



Safe to use
any prefix
length

✗ MATRYOSHKA UNAWARE

Trained **only for full length**.

Full Vector (768 dims)



Prefix (512 dims)



Prefix (256 dims)



Prefix (128 dims)



**QUALITY
DEGRADES AS
DIMENSIONS DROP**



Trained only
for full length
objective



Quality drops
with shorter
prefixes



Short prefixes
may not be
reliable

FP32 vs Matryoshka 512 (MRL-FP16-512) – Fidelity Dashboard

Top-10 Neighbor Analysis • 10,000 Queries • K = 10



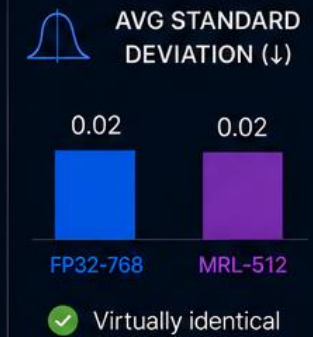
QUERY COUNT

10,000
queries

OVERALL FIDELITY (Top-10 Overlap)

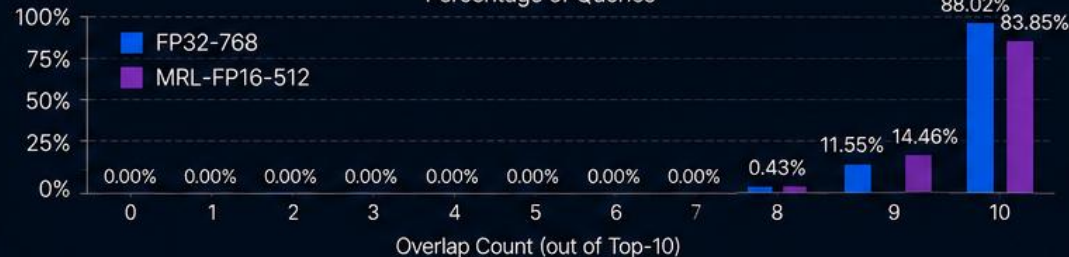
84.37%
Average Top-10 Overlap
Average Overlap Count: **8.4365**

DISTANCE METRICS (Top-10 Neighbors)



OVERLAP COUNT DISTRIBUTION

Percentage of Queries



✓ Distributions are highly similar with mass concentrated at 10.

HUBNESS SUMMARY (Top-10 Neighbors)

Maximum Occurrences (Higher = More Hubness)



✓ Hubness is similar with low maximum occurrences, indicating minimal hubness effect.



KEY TAKEAWAY

MRL-FP16-512 (Matryoshka 512) maintains very similar geometry compared to FP32-768 across all key distance metrics and overlap, with minimal change in hubness.



FIDELITY STATUS

✓ **GOOD – HIGH FIDELITY**

FP32 vs Matryoshka-512

Same Data • Same Vector Index • Increasing Compression

TOTAL DATABASE SIZE

(Table + Vector Index)



FP32

16.39 GB

Baseline

MRL-512

4.98 GB

70% smaller
vs FP32

EMBEDDING STORAGE (TABLE)



FP32

15.45 GB

Baseline

MRL-512

4.03 GB

74% smaller
vs FP32



VECTOR INDEX SIZE

(DiskANN Index)

FP32

0.94 GB

Unchanged across
all representations

MRL-512

0.94 GB



ROW COUNT

FP32

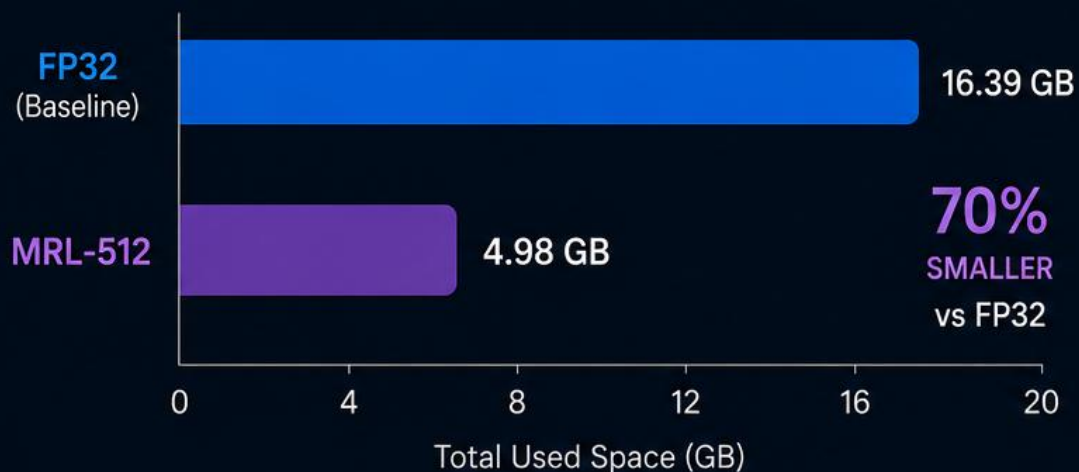
3,694,137

Same data across
all representations

MRL-512

3,694,137

STORAGE REDUCTION (Total Used Space)



KEY TAKEAWAYS



Matryoshka-512 reduces total database size by 70% with the same vector index.



Matryoshka-512 reduces embedding storage by 74%.



Vector index size remains constant (0.94 GB) across all representations.



Results averaged across 20 different queries with 3 runs each (60 executions per representation).

Only vector-index related tables included in logical reads:

Embeddings table + Worktable + Vector index graph edge table.

Environment: SQL Server



Why can't we stop here?

MATRYOSHKA 256

- Compression attempt 2

FP32 vs Matryoshka 256 – Fidelity Dashboard

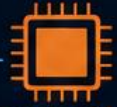
Top-10 Neighbor Overlap Analysis • 10,000 Queries

COMPARISON



FP32-768

VS



Matryoshka
FP16-256

QUERY COUNT



10,000

queries

OVERALL FIDELITY



72.04%

Average Top-10 Overlap

AVERAGE OVERLAP
COUNT (OUT OF 10)



7.2036

neighbors

AVERAGE TOP-K
OVERLAP PERCENT



MINIMUM OVERLAP
COUNT



1

neighbors

MAXIMUM OVERLAP
COUNT



10

neighbors

EXACT MATCH
QUERY COUNT



475

queries

EXACT MATCH
QUERY PERCENT



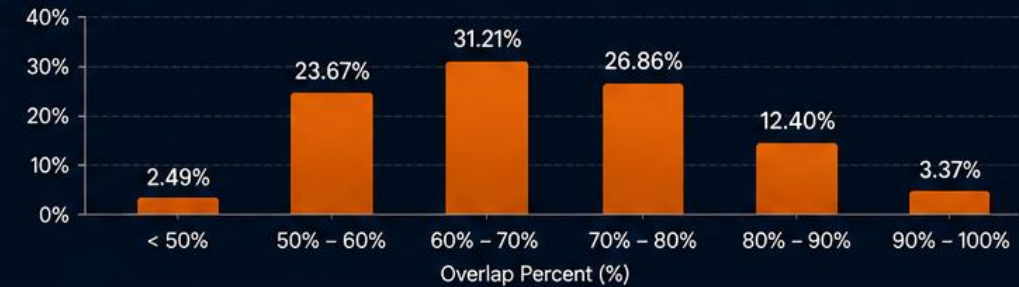
OVERLAP COUNT DISTRIBUTION

Percentage of Queries



OVERLAP PERCENT DISTRIBUTION

Percentage of Queries



KEY TAKEAWAY

Matryoshka 256 (FP16) preserves less of the embedding space compared to FP32-768. 72.04% average Top-10 overlap across 10,000 queries, with exact match for 4.75% of queries.



FIDELITY STATUS



FAIR – MODERATE FIDELITY

Noticeable deviation from FP32 neighborhood structure

Storage Comparison: FP32 vs Matryoshka256 (FP16)

Storage usage across tables and vector indexes

TABLE ROW COUNT



3,694,137

Rows (Both)

VECTOR INDEX ROW COUNT



3,694,137

Rows (Both)

TOTAL USED STORAGE



16.39 GB

FP32 | 2.96 GB

Matryoshka256 (FP16)

TOTAL RESERVED STORAGE



16.56 GB

FP32 | 2.96 GB

Matryoshka256 (FP16)

STORAGE REDUCTION (USED)



81.9%

Lower with Matryoshka256 (FP16)
(13.43 GB saved)

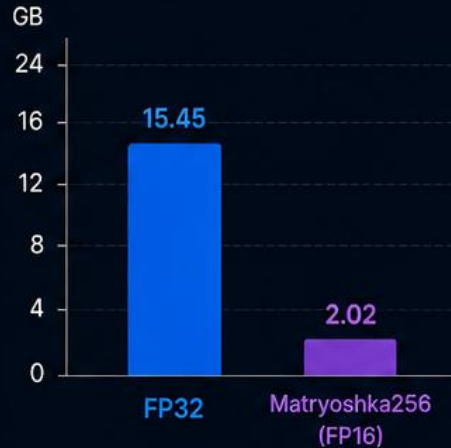
STORAGE REDUCTION (RESERVED)



82.1%

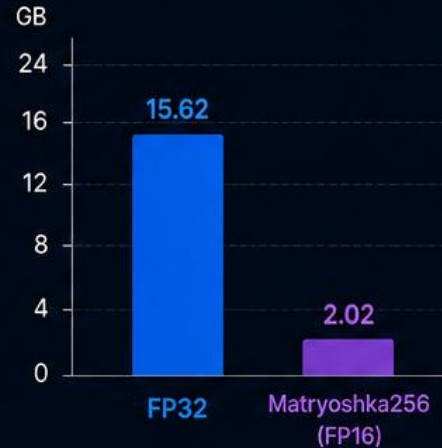
Lower with Matryoshka256 (FP16)
(13.60 GB saved)

TABLE USED SPACE (GB)



87.0% less table used space
(13.43 GB saved)

TABLE RESERVED SPACE (GB)



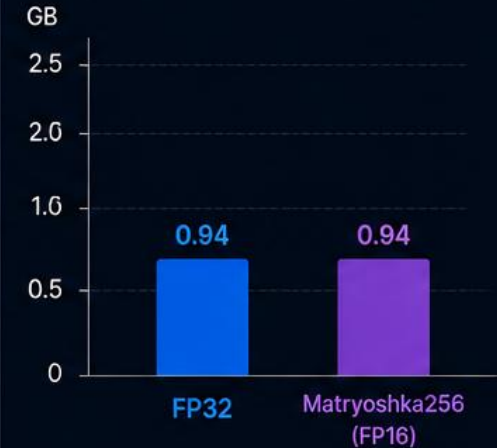
87.1% less table reserved space
(13.60 GB saved)

VECTOR INDEX USED SPACE (GB)



0.0% difference in index used space
(0.00 GB difference)

VECTOR INDEX RESERVED SPACE (GB)



0.0% difference in index reserved space
(0.00 GB difference)



FP32

Full Precision (768 dims)



Matryoshka256 (FP16)

256 dims

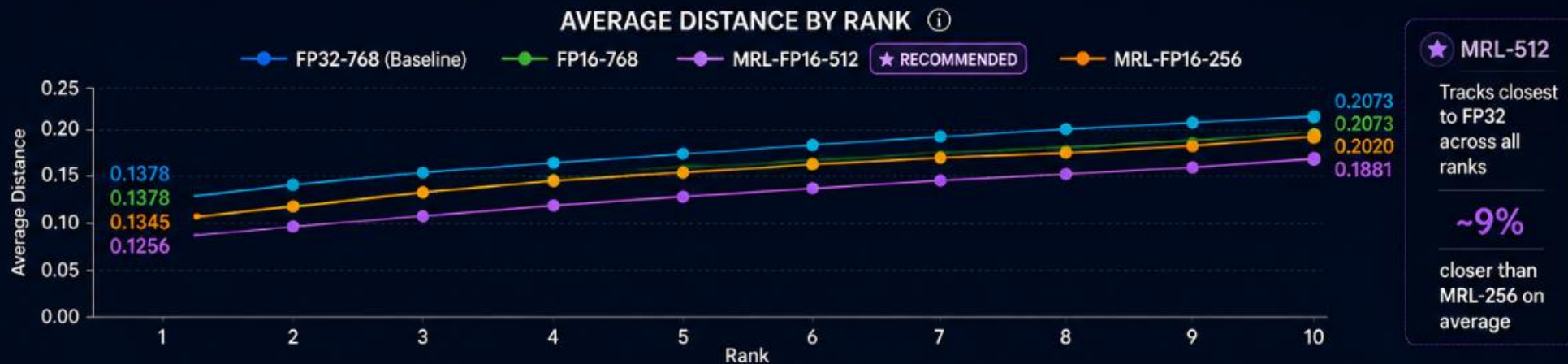


Lower values indicate better storage efficiency.

Summary across all 4
representations

GEOMETRY COMPARISON ACROSS REPRESENTATIONS

Run Summary • Top-10 Neighbors • 10,000 Queries



★ **MRL-512 BEST**
highest stability across the ranking



★ **MRL-512 HIGHEST**
most diversity in retrieved neighbors



★ **MRL-512 LOWER (BEST)**
less hubness, more balanced results



FINAL TAKEAWAY: **MRL-512** preserves geometry almost as well as **FP32**.



Closest to FP32
across all ranks



Best stability
(Rank 1 as % of Rank 10)



Most unique neighbors
(highest diversity)



Lowest hubness
(lower is better)

512 vs 256

- ✓ ~9% closer (lower distance)
- ✓ More diverse neighbors
- ✓ Less hubness (4 vs 5)

SPACE COMPARISON ACROSS REPRESENTATIONS

Same Data • Same Index Type • Same Row Count (3,694,137)



TOTAL ROWS

3,694,137

Identical across all representations



TOTAL TABLES

4

4 Representations Compared



TOTAL USED SPACE RANGE

2.96 GB – 16.39 GB

5.5x Difference



TOTAL RESERVED SPACE RANGE

2.96 GB – 16.56 GB

5.6x Difference

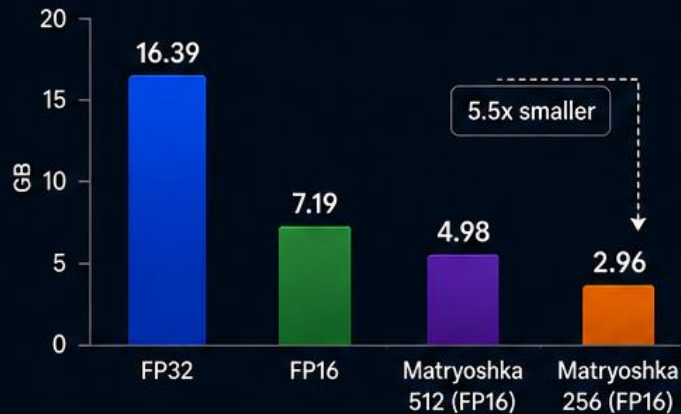


VECTOR INDEX USED

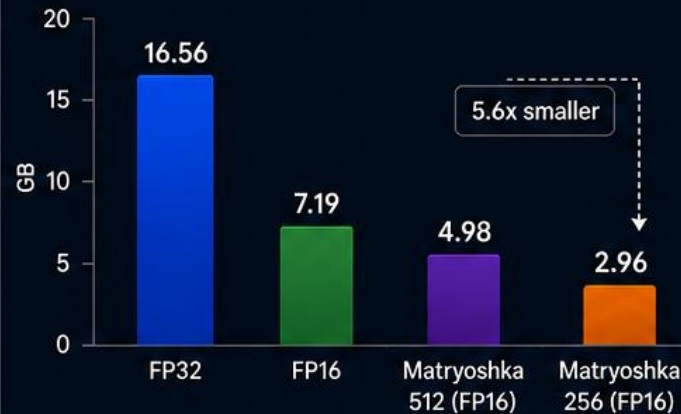
0.94 GB

Same across all representations

TOTAL USED SPACE (GB) ⓘ



TOTAL RESERVED SPACE (GB) ⓘ



SPACE REDUCTION vs FP32 ⓘ
(BY TOTAL USED GB)



BREAKDOWN OF TOTAL USED SPACE (GB) ⓘ

■ Table Used (GB) ■ Vector Index Used (GB)

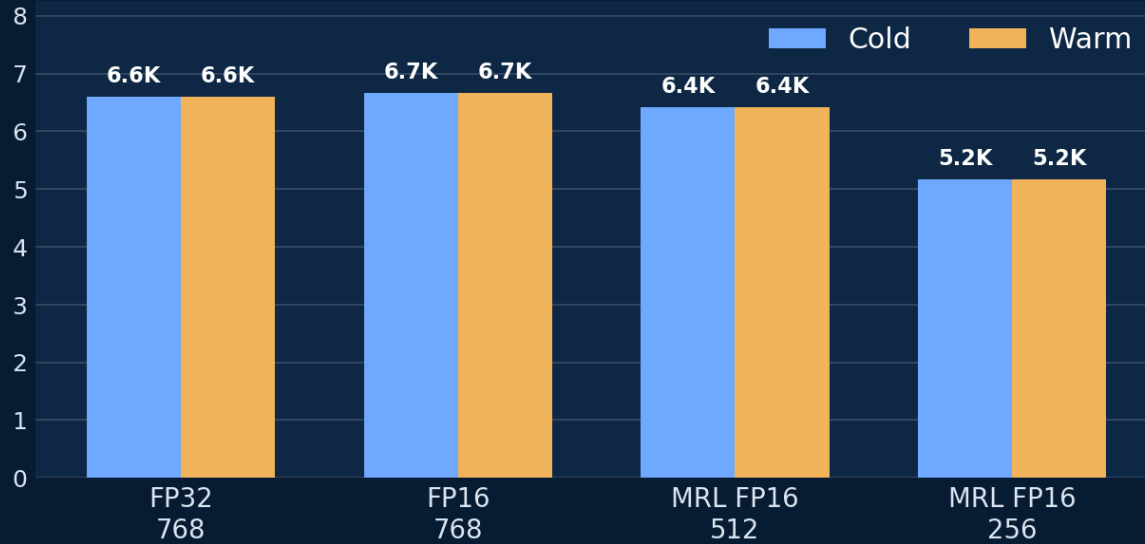


TAKEAWAY: MRL-256 is **5.5x smaller** than FP32 with strong fidelity.

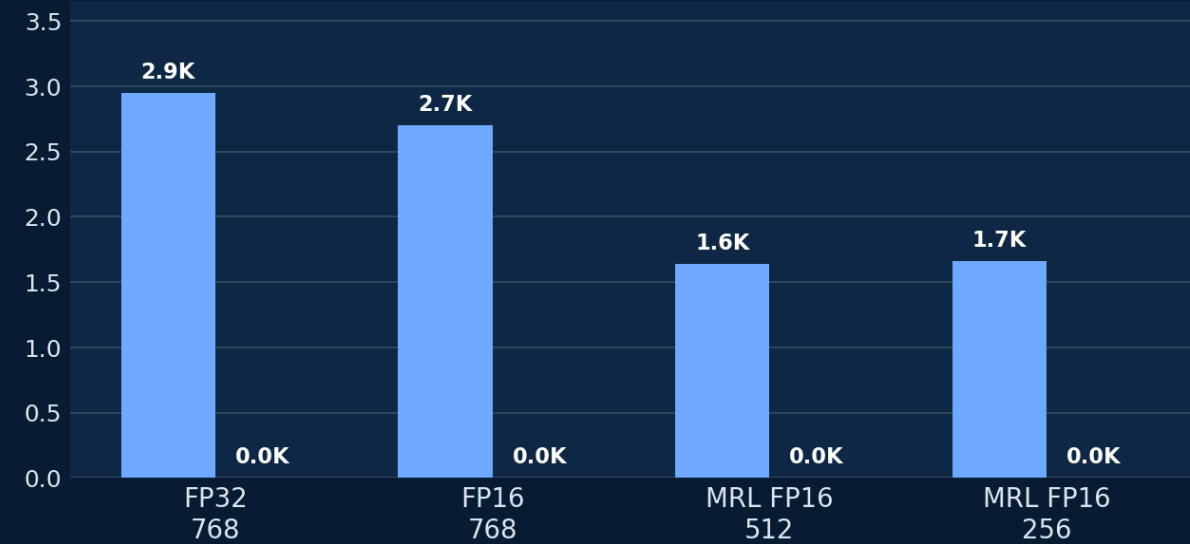
Indexed Vector Search: Cold vs Warm

20 queries • embedding table + vector index • average per query

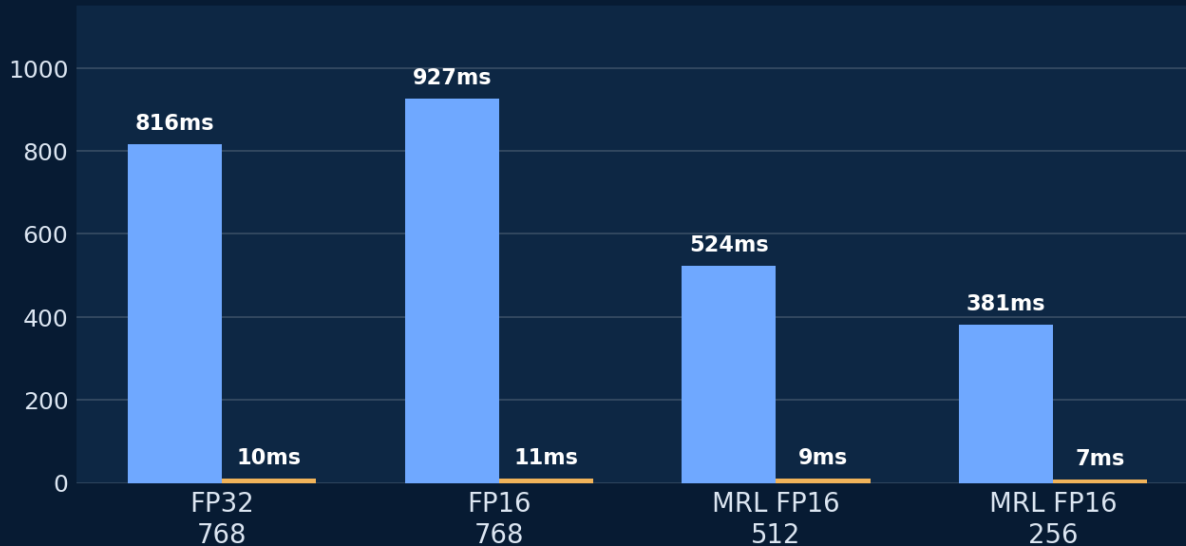
Logical Reads



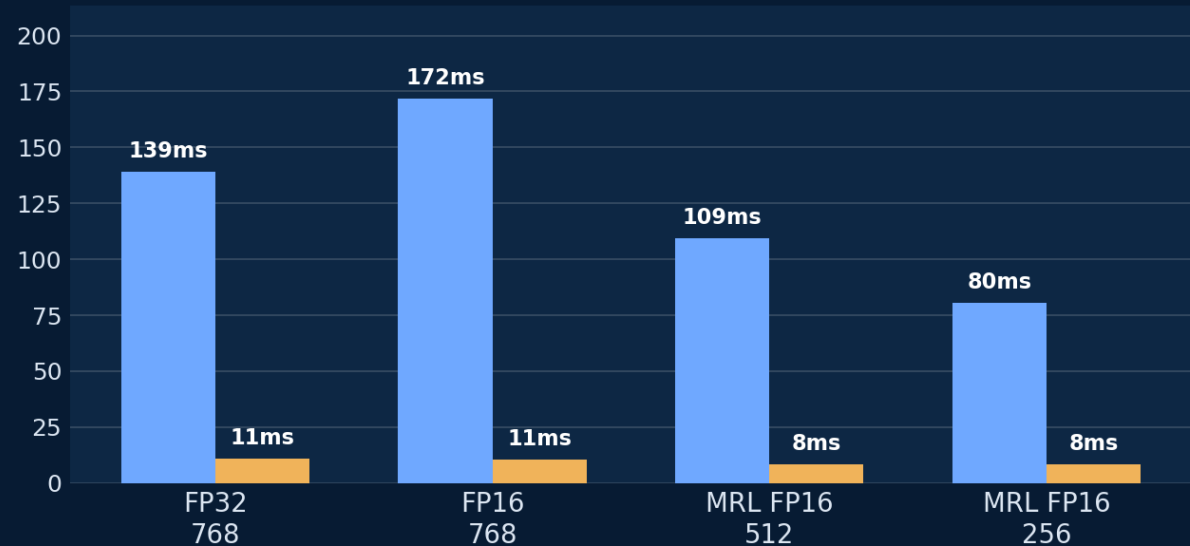
Physical Reads



Elapsed Time



CPU Time

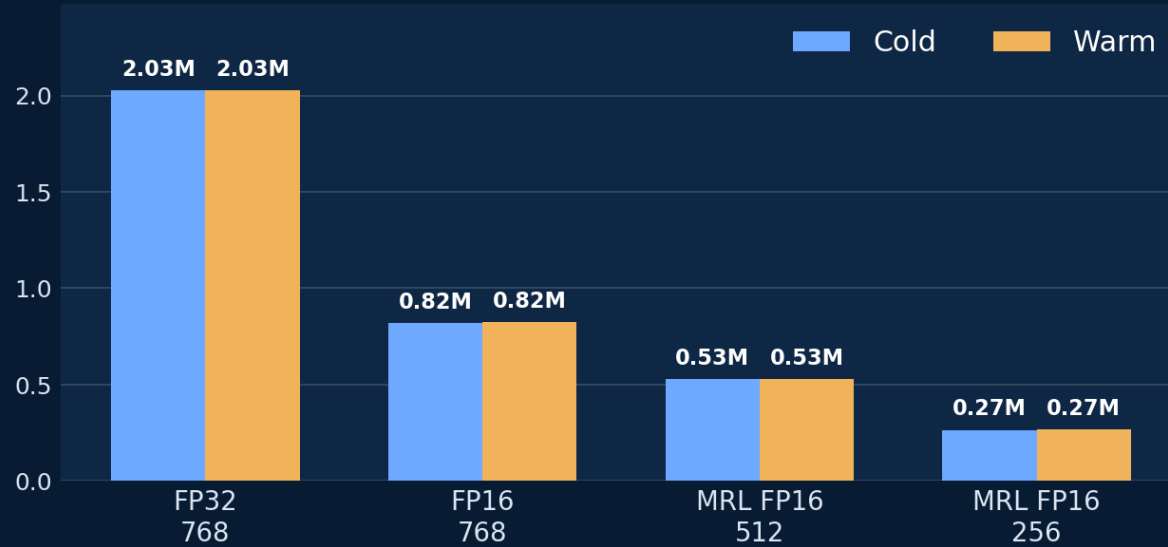


Warm cache removes physical I/O; logical traversal work stays the same.

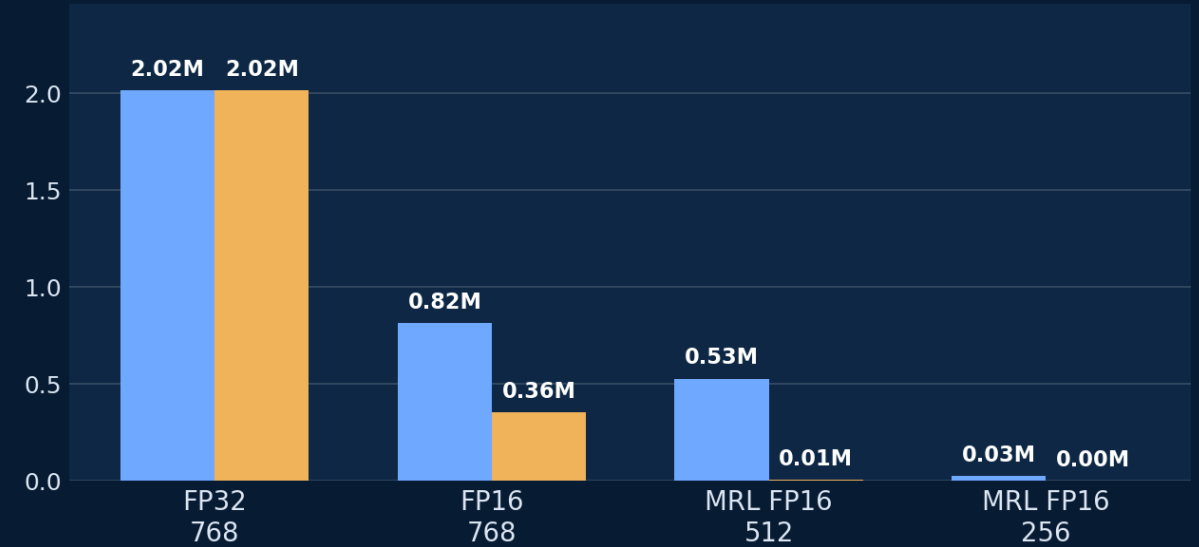
Exact Search (Table Scan): Cold vs Warm

20 queries • embedding table only • average per query

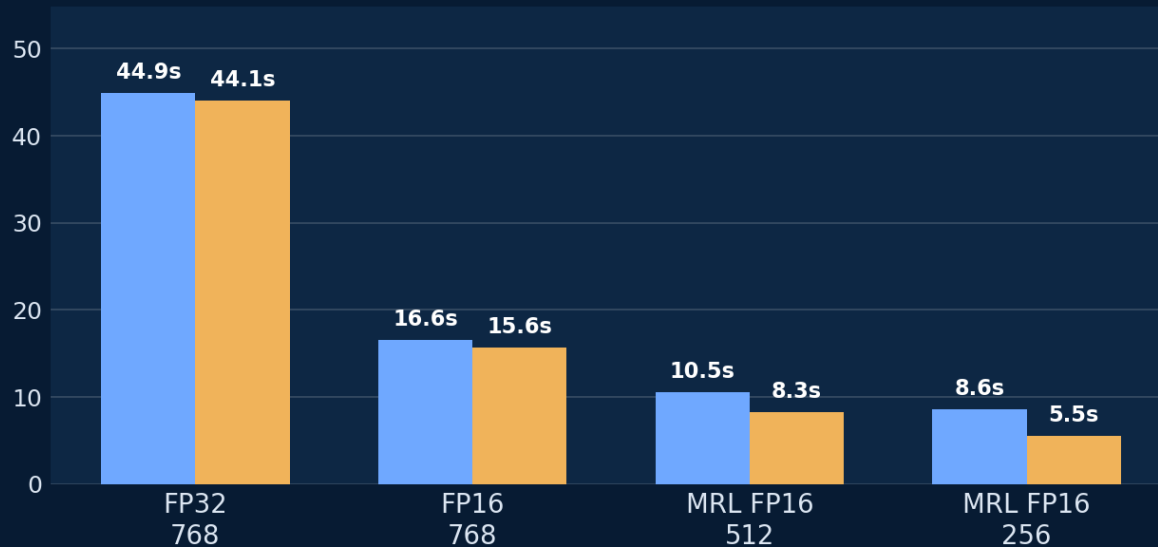
Logical Reads



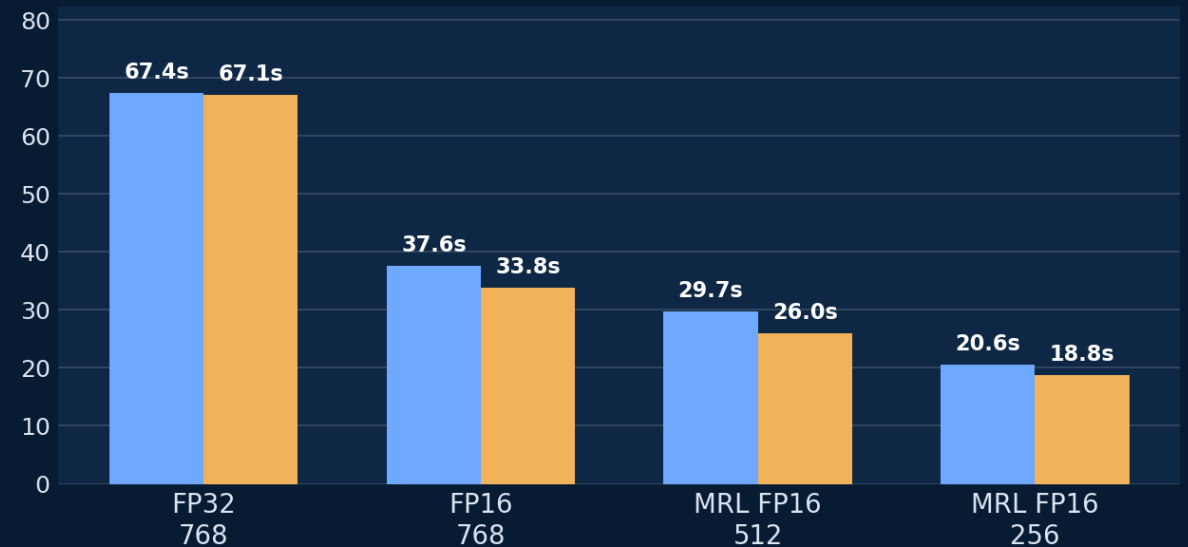
Read-Ahead Reads



Elapsed Time



CPU Time



Smaller representations reduce scan work; warm cache mainly improves runtime, not logical work.

Tradeoffs with Vector Index

Pros



Performance Gains



Cons



Approximate



Not updateable



Can't be optimized for space



Takeaway: Pragmatic Implementation of Vector search:
Higher compression, some tradeoff with quality, table scans.

Vector Representation Scoreboard

METRIC	 FP32 (BASELINE)	 FP16	 MATRYOSHKA (MRL) – 512	 MATRYOSHKA (MRL) – 256
 STORAGE (Compared to FP32)	— (Baseline)	50% smaller ↓ (~50% reduction)	33% smaller ↓ (~33% reduction)	67% smaller ↓ (~67% reduction)
 MEMORY (Per Vector)	4 bytes × 768 (Highest)	2 bytes × 768 ↓ (~50% less)	2 bytes × 512 ↓ (~33% less)	2 bytes × 256 ↓ (~67% less)
 DISTANCE COMPUTATION SPEED*	1.0 × (Baseline)	~1.8 × ↑ (Faster)	~2.4 × ↑ (Faster)	~3.5 × ↑ (Fastest)
 SEMANTIC NEIGHBORHOODS (Preservation)	 Excellent (Reference)	 Very Good	 Very Good (Designed)	 Good
 DISTANCE CONCENTRATION (Spread)	 Excellent (Wide Spread)	 Very Good (Wide Spread)	 Very Good (Wide Spread)	 Good (Some Concentration)
 HUBNESS (Risk)	 Low (Best)	 Low	 Low (Very Low)	 Low-Medium (Slight Increase)
 TYPICAL OUTCOME	Highest quality, highest storage and compute	Great quality with large storage savings	Excellent trade-off of quality, storage and speed	Maximum savings and speed; small quality trade-off

* Relative to FP32 (cosine similarity on same hardware). Actual numbers vary by system.

DECISION MATRIX

Decision	Owner	Recommendation	Why it Matters
Embedding model	 ML Team	Choose a model trained for your domain (e.g., Nomic, OpenAI, Cohere, etc.)	Determines semantic quality before compression begins.
Matryoshka-aware model?	 ML Team	Prefer Matryoshka-trained models if dimensionality reduction is expected.	Enables safe reduction without retraining or post-processing.
Embedding precision (FP32 vs FP16)	 ML + DBA	FP16 is usually the best first optimization.	~50% storage reduction with minimal quality loss.
Embedding dimensions (768 vs 512 vs 256)	 ML + DBA	Validate progressively using geometry tests.	Smaller isn't always better; geometry determines search behavior.
Quality acceptance criteria	 ML Team	Define acceptable overlap/recall for the application.	The database cannot determine whether quality is "good enough."
Storage & Index sizing	 DBA	Measure table size, vector index size and memory footprint.	Determines operational cost and scalability.
Query performance	 DBA	Measure latency, logical reads, cache behavior and throughput.	Confirms production performance.
Geometry validation	 ML + DBA	Measure semantic neighborhoods, distance concentration and hubness.	Ensures compression preserves the structure of the embedding space.
Production recommendation	 Joint Decision	Select the smallest representation that passes the quality gates.	Balances quality, storage and performance.

FINAL TAKEAWAYS



FP16
is the easiest win.

Best speed and efficiency improvements with minimal trade-offs.



Matryoshka
enables safe
dimensionality reduction.

Multi-length representations maintain fidelity across different vector sizes.



Geometry
determines whether
compression is
acceptable.

Preserve semantic neighborhoods and structure to ensure search quality and performance.

#References

1. **Manning, C. D., Raghavan, P., & Schütze, H.** *Introduction To Information Retrieval*. Cambridge University Press.
<https://Nlp.Stanford.Edu/Ir-book/>
2. **Kusupati, A., Et Al.** (Neurips 2022). *Matryoshka Representation Learning*.
<https://Arxiv.Org/Abs/2205.13147>
3. **Nomic AI.** *Nomic-embed-text-v1.5 Model Card*.
<https://Huggingface.Co/Nomic-ai/Nomic-embed-text-v1.5>
4. **Microsoft Research.** *Diskann: Fast Accurate Billion-point Nearest Neighbor Search*.
<https://Github.Com/Microsoft/Diskann>
5. **Microsoft Learn.** *SQL Server 2025 AI & Vector Search Documentation*.
<https://Learn.Microsoft.Com/Sql/Sql-server/Ai/>
6. **Aggarwal, C. C., Hinneburg, A., & Keim, D. A.** (2001). *On The Surprising Behavior Of Distance Metrics In High Dimensional Space.*
7. **Radovanović, M., Nanopoulos, A., & Ivanović, M.** (2010). *Hubs In Space: Popular Nearest Neighbors In High-dimensional Data*. *Journal Of Machine Learning Research*, 11, 2487–2531.
8. **Lian Zhou.** *Next Generation Vector Databases* (Book).
(Recommended Reading On Modern Vector Database Architectures And ANN Indexing.)
9. **Joe Sack.** *SQL Server Vector Search Essentials*. Sqlskills Training. [Sqlskills Vector Search Essentials](https://Www.Sqlskills.Com/Sql-server-training/Aivse/?Utm_source=chatgpt.Com) ([Sqlskills][1])
10. **Joe Sack.** *Vector Search In Practice*. Sqlskills Training. [Sqlskills Vector Search In Practice](https://Www.Sqlskills.Com/Product/Aivsp/?Utm_source=chatgpt.Com) ([Sqlskills][2])



HOW I USED AI FOR THIS TALK

Suggestions on open source models to use

Debugging Python for embedding generation

Drawing dashboards based on provided #s

Basic queries to refine on and work further

Imagery to explain concepts with provided input

Thank You

EIGHTKB